

Compute, Storage, Networking and the Deployment Landscape

A working map of Google Cloud for the service you are about to ship

WHAT WE'LL COVER

Eight chapters, five labs

01

Landing in the sandbox

02

The compute spectrum

03

Compute Engine and instance groups

04

Containers, Kubernetes and serverless

05

Choosing, and the cost of hybrid

06

Storage and databases

07

Networking: reaching and protecting the service

08

Artifacts and the Claude-based service

01

Landing in the sandbox

Open Cloud Shell, point it at your sandbox project, and prove every tool we need today is there.

01 Landing in the sandbox

02 The compute spectrum

03 Compute Engine and instance groups

04 Containers, Kubernetes and serverless

05 Choosing, and the cost of hybrid

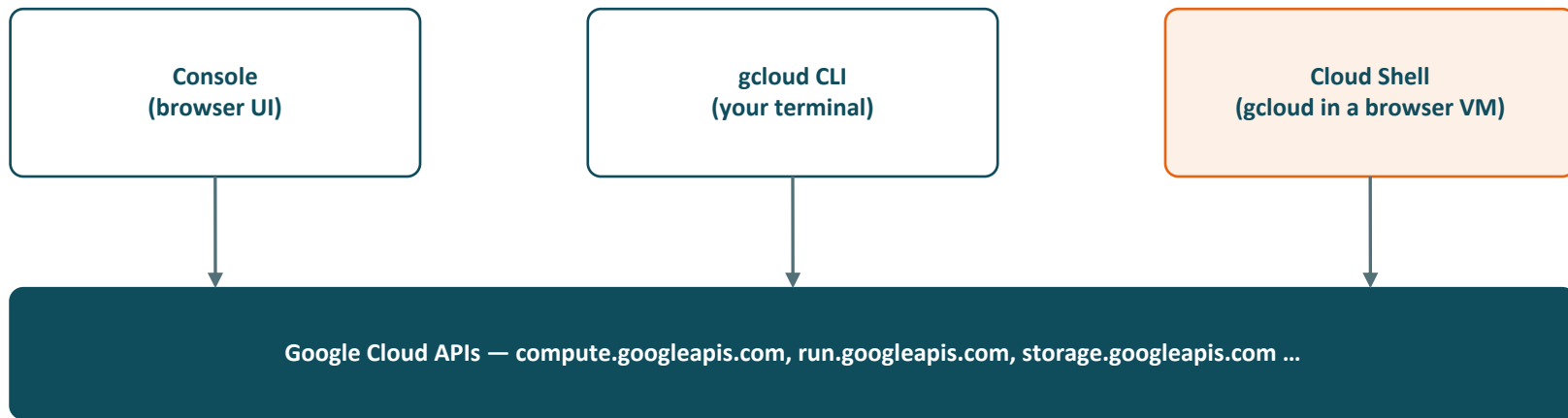
06 Storage and databases

07 Networking: reaching and protecting the service

08 Artifacts and the Claude-based service

THREE DOORS, ONE API

Console, Cloud Shell and gcloud talk to the same control plane

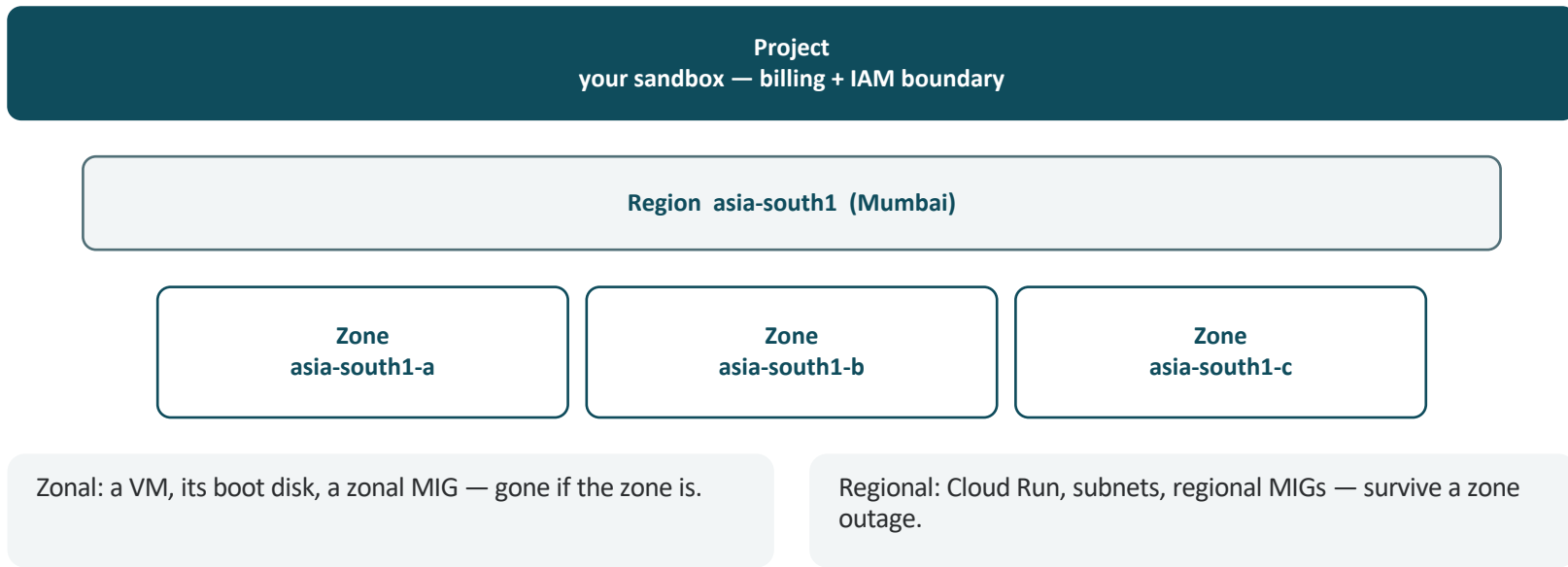


Same identity (your Google account), same permissions (IAM), same audit log. Cloud Shell is the door we use today: nothing to install, one tool version for the whole room, ~5 GB of home directory that persists between sessions.

THE RULE If you can click it, you can script it — learn the gcloud form, the console will always be there.

PROJECT, REGION, ZONE

Every resource has an address: project → region → zone



THE RULE

Decide the region once, early — data, VMs and services in different regions pay latency and egress.

Set the project, enable the APIs, check the tools

HANDS-ON · LAB 0

1

Open the console, pick your sandbox project, click the Cloud Shell icon

console.cloud.google.com → icon top right; the prompt ends with (your-project-id)

2

Point gcloud at the project and the region

gcloud config set project YOUR_PROJECT_ID · gcloud config set compute/region asia-south1 ·
gcloud config set compute/zone asia-south1-a

3

Enable the six APIs used today (one command)

gcloud services enable compute.googleapis.com run.googleapis.com cloudbuild.googleapis.com
artifactregistry.googleapis.com secretmanager.googleapis.com storage.googleapis.com

4

Verify the toolchain

gcloud --version | head -1 · docker --version · python3 --version · kubectl version --client |
head -1

Console path: APIs & Services → Enable APIs and services → search each API → Enable. Nothing here costs money; if a step fails, raise a hand.

EXPECTED

```
$ gcloud config set project sandbox-rohan-07
Updated property [core/project].
$ gcloud services enable
compute.googleapis.com \
  run.googleapis.com \
  cloudbuild.googleapis.com \
  artifactregistry.googleapis.com \
  secretmanager.googleapis.com \
  storage.googleapis.com
Operation "operations/acat.p2-6..." finished
successfully.
$ gcloud --version | head -1
Google Cloud SDK 583.0.0
$ python3 --version
Python 3.13.5
```

THE RULE

One project, one region, six APIs — check the prompt shows your project id before every lab.

02

The compute spectrum

Four ways to run the same code — what you control, what Google runs for you, and what each step costs you.

- 01 Landing in the sandbox
- 02 The compute spectrum**
- 03 Compute Engine and instance groups
- 04 Containers, Kubernetes and serverless
- 05 Choosing, and the cost of hybrid
- 06 Storage and databases
- 07 Networking: reaching and protecting the service
- 08 Artifacts and the Claude-based service

THE COMPUTE SPECTRUM

Four ways to run the same code — control traded for convenience

Virtual machine

Compute Engine

You own: OS, patches, runtime, process, scaling

Google owns: hardware, hypervisor, network

Container

Cloud Run / GKE

You own: image, app config

Google owns: OS, runtime host, scheduling

Kubernetes

GKE

You own: manifests, cluster shape (Standard) or just workloads (Autopilot)

Google owns: control plane, node upgrades

Serverless

Cloud Run, Cloud Run functions

You own: code + a container or a function

Google owns: everything else, incl. scale-to-zero

more control, more to operate

less to operate, fewer knobs

THE RULE

Start as far right as the workload allows; move left only for a reason you can name.

WHAT "MANAGED" MEANS

The same web app, four ways — what is left on your plate

Responsibility	VM (Compute Engine)	Container (Cloud Run)	Kubernetes (GKE Autopilot)	Function (Cloud Run functions)
OS patching	you	Google	Google	Google
Runtime install (Python, Java...)	you	you (image) / buildpacks	you (image)	Google (base image)
Process supervision, restarts	you (systemd)	Google	Google (kubelet)	Google
HTTPS endpoint + certificate	you (LB + cert)	Google (run.app URL)	you (Ingress / Gateway)	Google
Scaling	you (MIG + autoscaler)	Google (per request)	Google (HPA you configure)	Google (per event)
Scale to zero	no	yes	no (Autopilot bills per pod)	yes
Logs + metrics	you (install agent)	Google	Google	Google

THE RULE

Count the "you" cells in a column — that is the operations team you are hiring, whether or not you meant to.

Where would you put it?

Same generic company, four workloads. Pick a rung for each and be ready to say why.

A) Nightly report

Runs once a day for 20 minutes, reads the database, writes a PDF to a bucket.

B) Public API, 24×7

Steady 200 req/s, spiky at 9 am, must answer in < 300 ms.

C) Licensed legacy app

Vendor binary, needs a specific kernel module and a static IP for its licence server.

D) Image-resize on upload

Fires when a file lands in a bucket; 50 ms of work; thousands of times a day, zero at night.

What you give up moving right — and why that is usually fine

MOVING RIGHT COSTS

- Cold starts when scaling from zero (hundreds of ms to seconds)
- Request/response model: long-lived connections and background threads need care
- Ephemeral local disk — state must live elsewhere (bucket, database)
- Networking is opt-in: no VPC by default, egress via Direct VPC egress when needed
- Fewer knobs: no kernel modules, no custom OS agents

MOVING RIGHT BUYS

- Zero patching, zero process supervision, zero capacity planning
- Scale to zero: an idle service costs nothing
- HTTPS, revisions, rollbacks and traffic splitting built in
- Logs and metrics with no agent to install
- One deploy command from source to URL

THE RULE

A stateless HTTP service with a managed database is the common case — and the right edge was built for exactly that.

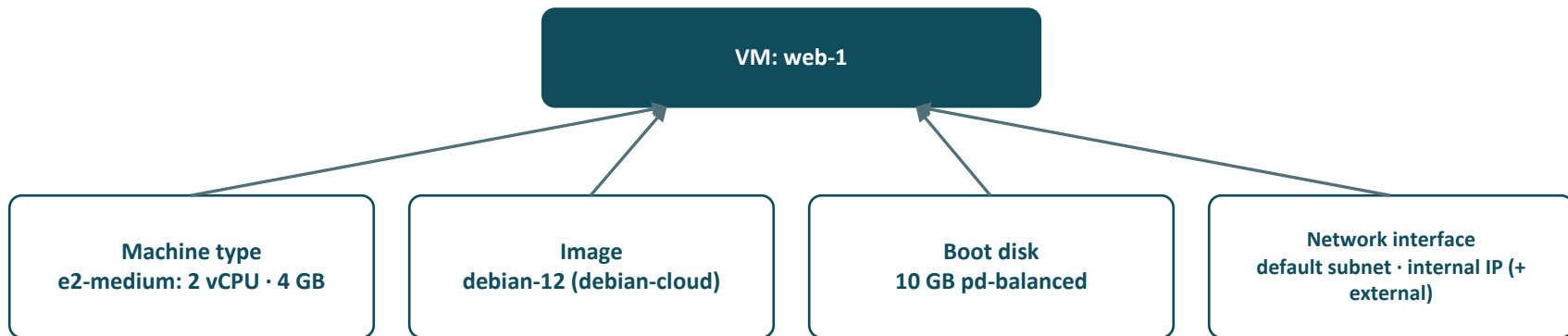
03

Compute Engine and instance groups

Virtual machines the Google way: machine families, images, disks — and turning one VM into a fleet that scales itself.

- 01 Landing in the sandbox
- 02 The compute spectrum
- 03 Compute Engine and instance groups**
- 04 Containers, Kubernetes and serverless
- 05 Choosing, and the cost of hybrid
- 06 Storage and databases
- 07 Networking: reaching and protecting the service
- 08 Artifacts and the Claude-based service

Anatomy of a VM: machine type, image, disk, network interface



Hanging off those four: metadata and startup scripts (what runs at boot), a service account (what the VM may call), network tags (which firewall rules apply), labels (billing). Regional vs zonal: a VM is always zonal; its boot disk lives in the same zone.

THE RULE

Machine type, image, disk, network — decide those four and the VM is decided.

Machine families — and how to read e2-standard-4

e2 - standard - 4

└─ 4 vCPUs
└─ memory ratio: standard 4 GB/vCPU · highmem 8 · highcpu 1-2
└─ series: E2 (family: general-purpose, cost-optimised)

Family	Series	Use it for	Note
General-purpose, cost-optimised	E2 (incl. shared-core e2-micro / small / medium)	Web servers, dev/test, small services — today's labs	Cheapest; CPU may be shared on micro/small
General-purpose, current gen	N4 · N4A (Arm/Axion) · N4D	Most production services	Hyperdisk only; better price/performance than N2
Performance	C4 · C4A (Arm/Axion) · C4D · C3	Latency-sensitive APIs, databases, CI runners	Highest all-core turbo; more expensive
Specialised	M (memory) · Z (storage) · A/G (GPU/accelerator)	In-memory DBs, big local SSD, ML training/inference	Quota needs requesting in advance

THE RULE

Default to E2 for anything small, N4 for anything real, and let a benchmark — not a blog post — move you to C4.

Image families: ask for debian-12, get today's patched build

WORKED EXAMPLE — IMAGE FAMILY RESOLVES TO A DATED BUILD

```
$ gcloud compute images describe-from-family debian-12 \
  --project=debian-cloud --format='value(name,creationTimestamp)'
debian-12-bookworm-v20260822 2026-08-22T09:14:03.201-07:00

$ gcloud compute images list --filter='family~^debian' \
  --format='table(name,family,project)' | head -4
NAME                                FAMILY    PROJECT
debian-11-bullseye-v20260822        debian-11 debian-cloud
debian-12-bookworm-v20260822        debian-12 debian-cloud
debian-13-trixie-v20260822          debian-13 debian-cloud
```

Three kinds of image

- Public — Google-maintained OS builds (debian-cloud, ubuntu-os-cloud, cos-cloud...)
- Custom — you bake your app in once; every VM from it boots ready
- Marketplace — vendor images (licensed software), billed per hour

Family = pointer to the newest image of a line. Templates should reference a family; audits should reference a dated image name.

THE RULE

Reference the family in templates, the dated image in audits — never a hard-coded image name in a script you will run next month.

Disks: Persistent Disk, Hyperdisk, Local SSD

	Persistent Disk	Hyperdisk	Local SSD
What it is	Network block storage (pd-standard / pd-balanced / pd-ssd)	Next-gen network block storage; IOPS/throughput set independently of size	NVMe flash on the host
Survives VM stop / delete	Yes (detachable, snapshot-able)	Yes (detachable, snapshot-able)	No — scratch only
Zonal / regional	Both (regional = 2-zone replica)	Zonal, some regional	Zonal, bound to the host
Works with	E2, N2, N2D, C3 ...	N4, C4, C4A, C3 ... (N4/C4: Hyperdisk only)	Series with local SSD slots
Typical use	Boot disks, general data	Databases, high-throughput apps	Caches, temp files, shuffle space

Backups: gcloud compute snapshots create — incremental, stored in Cloud Storage, restorable into any zone.

THE RULE

If the data must outlive the VM, it lives on a Persistent Disk or Hyperdisk — Local SSD is a cache, never a home.

COMPUTE ENGINE

Worked example: one VM from the CLI, and what the output says

WORKED EXAMPLE — CREATE A VM

```
$ gcloud compute instances create web-1 \
  --machine-type=e2-micro \
  --image-family=debian-12 --image-project=debian-cloud \
  --boot-disk-size=10GB --boot-disk-type=pd-balanced \
  --tags=http-server \
  --metadata-from-file=startup-script=startup.sh
Created [https://www.googleapis.com/compute/v1/projects/sandbox-rohan-07/
zones/asia-south1-a/instances/web-1].
```

NAME	ZONE	MACHINE_TYPE	PREEMPTIBLE	INTERNAL_IP	EXTERNAL_IP	STATUS
web-1	asia-south1-a	e2-micro		10.160.0.2	34.93.118.204	RUNNING

--tags=http-server
firewall rules target this tag, not the VM name

--metadata-from-file=startup-script
runs as root at every boot — install, configure, start

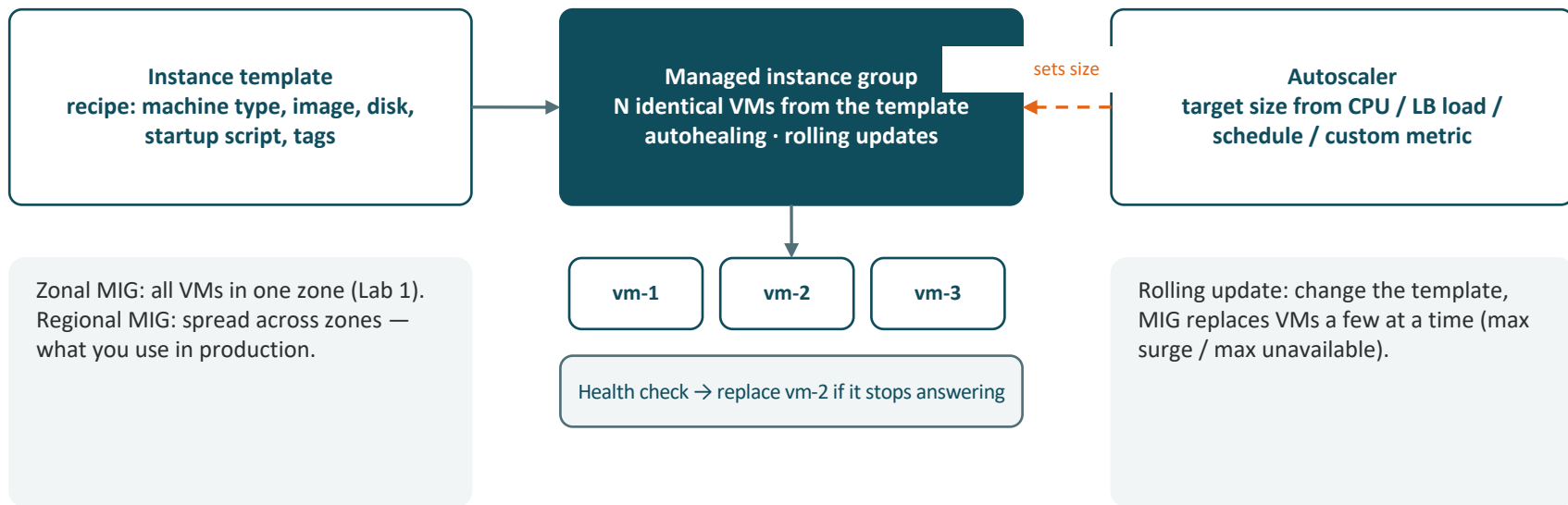
INTERNAL_IP always · EXTERNAL_IP only
because we did not pass --no-address

THE RULE

A VM is a command, not a wizard — put the command in a file and you already have your first template.

MANAGED INSTANCE GROUPS

From one VM to a fleet: template → group → autoscaler



THE RULE

Never hand-build the second VM — make a template, let the group stamp them out and keep them healthy.

AUTOSCALING

Worked example: CPU autoscaling, and why the cooldown matters

WORKED EXAMPLE — ATTACH AN AUTOSCALER

```
$ gcloud compute instance-groups managed set-autoscaling web-mig \
  --zone=asia-south1-a --min-num-replicas=1 --max-num-replicas=3 \
  --target-cpu-utilization=0.6 --cool-down-period=60
Created [https://www.googleapis.com/compute/v1/projects/
  sandbox-rohan-07/zones/asia-south1-a/autoscalers/web-mig-xkq2].
---
autoscalingPolicy:
  coolDownPeriodSec: 60
  cpuUtilization: {utilizationTarget: 0.6}
  maxNumReplicas: 3
  minNumReplicas: 1
  mode: ON
  name: web-mig-xkq2
  target: ../instanceGroupManagers/web-mig
```

The four knobs

- min / max — the floor and the ceiling; min ≥ 2 for anything you cannot afford to lose
- target — the utilisation you are steering toward, not a threshold
- cooldown — ignore a new VM while it boots, or you scale out twice
- scale-in is slow on purpose (10-min window): dips do not delete VMs

THE RULE

Autoscaling is a controller, not a switch: set the target you want, the bounds you can afford, and a cooldown as long as a boot.

A VM, then a fleet — create, template, group, autoscale, tear down

HANDS-ON · LAB 1

1

Unzip the kit and run steps 1–3 of lab1.sh as they are

Bookshelf's first home: one e2-micro from debian-12 with startup.sh; a firewall rule for tag http-server

2

Open `http://EXTERNAL_IP` in a tab

You should see "Bookshelf baseline — Served by VM: bookshelf-vm" (give the startup script 60 s)

3

Fill TODO 1.4 — the instance template

Same recipe as the VM but e2-small; reference the image FAMILY, not a dated image

4

Fill TODO 1.5 — a zonal MIG of 2 from that template

--template, --size, --zone

5

Fill TODO 1.6 — autoscaling 1..3 on 60 % CPU, 60 s cooldown

Then list-instances; predict what the group size will be in 10 minutes

6

bash teardown.sh — before the break

Deletes the MIG, template, VM and firewall rule; safe to re-run

THE BLANKS

```
# TODO(lab-1.4)
gcloud compute instance-templates create
bookshelf-tmpl \
  --machine-type=_____ \
  --image-family=_____ --image-
project=_____ \
  --tags=http-server \
  --metadata-from-file=startup-
script=startup.sh

# TODO(lab-1.6)
--min-num-replicas=_____ --max-num-
replicas=_____ \
  --target-cpu-utilization=_____ --cool-
down-period=_____
```

Answers on the next slide. Console path: Compute Engine
→ VM instances → Create; Instance templates → Create;
Instance groups → Create (autoscaling section).

THE RULE

Template, group, autoscaler — three commands, and the second VM is never built by hand again.

What you should have seen — and the answers to the blanks

DEBRIEF · LAB 1

ANSWERS 1.4 · 1.5 · 1.6

```
--machine-type=e2-small
--image-family=debian-12 --image-project=debian-cloud
--template=bookshelf-tmpl --size=2 --zone=$ZONE
--min-num-replicas=1 --max-num-replicas=3
--target-cpu-utilization=0.6 --cool-down-period=60
```

PRODUCTION WOULD ADD

- Regional MIG (3 zones) · min 2 · a health check for autohealing
- A load balancer in front — chapter 7

EXPECTED — STEP 7, THEN AGAIN LATER

```
$ gcloud compute instance-groups managed list-instances \
    bookshelf-mig --zone=asia-south1-a
```

NAME	ZONE	STATUS	ACTION
INSTANCE_TEMPLATE			
bookshelf-mig-4k7p	asia-south1-a	RUNNING	NONE
bookshelf-mig-x2mc	asia-south1-a	RUNNING	NONE

~10 minutes later, VMs idle, min=1:

```
bookshelf-mig-4k7p  asia-south1-a  RUNNING  NONE  bookshelf-tmpl
```

COMMON STUMBLER

- Page not loading: firewall tag mismatch, or the startup script is still installing python3

THE RULE

The autoscaler did what you told it, not what you meant — bounds and targets are the specification.

04

Containers, Kubernetes and serverless

Images, orchestration, GKE, Cloud Run and Cloud Run functions: the managed rungs, and when each one is warranted.

- 01 Landing in the sandbox
- 02 The compute spectrum
- 03 Compute Engine and instance groups
- 04 Containers, Kubernetes and serverless**
- 05 Choosing, and the cost of hybrid
- 06 Storage and databases
- 07 Networking: reaching and protecting the service
- 08 Artifacts and the Claude-based service

CONTAINERS

An image is a filesystem plus a start command, built in layers

WORKED EXAMPLE — A DOCKERFILE FOR A SMALL FLASK SERVICE

```
FROM python:3.13-slim          # layer 1: base OS + Python
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt # layer 2: deps
COPY app.py .                  # layer 3: your code (changes most)
RUN useradd --create-home appuser
USER appuser                   # never run as root
ENV PORT=8080
CMD ["sh", "-c", "gunicorn --bind :$PORT --workers 1 --threads 8 app:app"]
```

Image = layers + metadata (CMD, PORT). Container = a process started from an image, isolated by the kernel — millisecond start, no OS of its own. Registry = where images live (Artifact Registry, chapter 8).

your code (app.py)

dependencies (pip install)

python:3.13-slim

debian base layers

top layer rebuilds every commit; lower layers are cached and shared

THE RULE

Pin the base image, run as non-root, copy code last — the image is your deployable unit from here on.

WHY ORCHESTRATION EXISTS

One container is easy; fifty on twenty machines is scheduling

- | | | |
|---|--------------------------------|---|
| 1 | Scheduling | which machine runs which copy, given CPU/RAM |
| 2 | Self-healing | restart or replace a copy that dies; drain a bad node |
| 3 | Scaling | more copies on load, fewer when idle |
| 4 | Rollout | v1 → v2 gradually, roll back on failure |
| 5 | Discovery & routing | copies find each other; the outside finds the service |

	Kubernetes (GKE)	Cloud Run
Scheduling	scheduler + node pools	hidden
Self-healing	kubelet restarts pods	hidden
Scaling	HPA (you configure)	per request, to zero
Rollout	Deployment strategy	revisions + traffic split
Discovery	Services, Ingress/Gateway	URL per service

THE RULE

Every platform to the right of a VM is answering these five questions — the difference is how much of the answer you must write.

What GKE gives you — and when a cluster is warranted

	Autopilot (recommended)	Standard
Nodes	Google-managed; you never see VMs	You size and manage node pools
Billing	Per pod CPU/RAM requested	Per node (VM price) + cluster fee
Upgrades, patching	Google, automatic	You schedule (or auto-upgrade channel)
Use when	Default for any new cluster	GPUs/TPUs with custom drivers, DaemonSets, privileged pods, specific node config

Trainer demo on a pre-built cluster: `kubectl get nodes`, deploy the sample, scale it — no learner clusters today (5-minute create, quota, cost).

A CLUSTER IS WARRANTED WHEN

- Many services share one platform
- Stateful / non-HTTP / custom networking needs
- The team already runs Kubernetes

NOT WARRANTED WHEN

- One to three stateless HTTP services
- Nobody on the team has run a cluster
- "We might need it later"

THE RULE

GKE is the right answer to a platform question, and the wrong answer to a single-service question.

Worked example: an Autopilot cluster, a Deployment, a scale

TRAINER DEMO — AUTOPILOT CLUSTER (VERSIONS AND IPS WILL DIFFER ON THE DAY)

```
$ gcloud container clusters create-auto demo --region asia-south1
```

```
Creating cluster demo in asia-south1... done.
```

NAME	LOCATION	MASTER_VERSION	MASTER_IP	MACHINE_TYPE	NODE_VERSION	NUM_NODES	STATUS
demo	asia-south1	1.33.4-gke.1134000	34.93.201.17	e2-small	1.33.4-gke.1134000		RUNNING

```
$ kubectl create deployment web --image=us-docker.pkg.dev/google-samples/containers/gke/hello-app:2.0  
deployment.apps/web created
```

```
$ kubectl scale deployment web --replicas=3
```

```
deployment.apps/web scaled
```

```
$ kubectl get pods -o wide
```

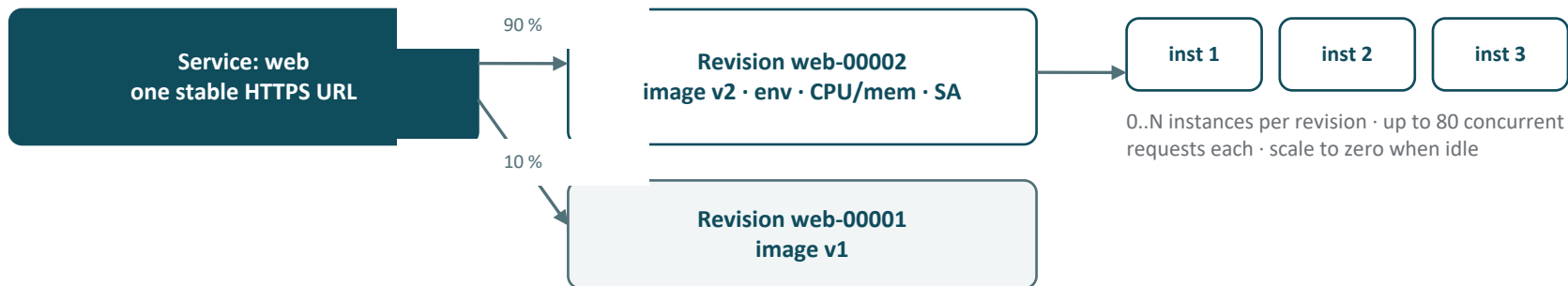
NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
web-6c9f8d7b4-2kx9p	1/1	Running	0	41s	10.68.0.12	gk3-demo-nap-1p2q...
web-6c9f8d7b4-7hrmz	1/1	Running	0	41s	10.68.1.4	gk3-demo-nap-8r3t...
web-6c9f8d7b4-ml4wc	1/1	Running	0	96s	10.68.0.11	gk3-demo-nap-1p2q...

Behind three commands: a Deployment object, a ReplicaSet, three Pods, node auto-provisioning — and a bill for the control plane whether or not anything runs.

THE RULE

Kubernetes answers the five questions in full — and asks you to describe every answer in YAML.

Serverless containers: service → revision → instance, from zero



Resource	For	Example
Service	request/response, HTTP or gRPC, scales per request	web front end, an API
Job	run to completion, retries, parallel tasks	nightly report, data migration
Worker pool	pull-based work with no endpoint	queue consumer

THE RULE

A revision is immutable and addressable — that is what makes rollback a traffic change, not a redeploy.

Worked example: deploy from source — no Dockerfile, one URL

WORKED EXAMPLE — GCP CLOUD RUN DEPLOY --SOURCE

```
$ gcloud run deploy web --source . --region asia-south1 --allow-unauthenticated
```

Deploying from source requires an Artifact Registry Docker repository to store built containers.

A repository named [cloud-run-source-deploy] in region [asia-south1] will be created.

Do you want to continue (Y/n)? y

Building using Buildpacks and deploying container to Cloud Run service [web] in project [sandbox-rohan-07] region [asia-south1]

✓ Building and deploying new service... Done.

✓ Uploading sources...

✓ Building Container... Logs are available at [https://console.cloud.google.com/cloud-build/builds/...].

✓ Creating Revision...

✓ Routing traffic...

✓ Setting IAM Policy...

Done.

Service [web] revision [web-00001-k7q] has been deployed and is serving 100 percent of traffic.

Service URL: <https://web-482915730261.asia-south1.run.app>

Buildpacks: no Dockerfile → language detected from requirements.txt / package.json / go.mod; a Procfile sets the start command. A Dockerfile in the folder wins if present.

THE RULE

From a folder of source to a public HTTPS URL is one command — the platform built, stored, deployed and routed.

Revisions and traffic splitting — rollout and rollback as routing

WORKED EXAMPLE — CANARY, PROMOTE, ROLL BACK

```
$ gcloud run deploy web --source . --region asia-south1 --set-env-vars APP_VERSION=v2
... Service [web] revision [web-00002-p2m] has been deployed and is serving 100 percent of traffic.
$ gcloud run services update-traffic web --region asia-south1 \
  --to-revisions=web-00002-p2m=10,web-00001-k7q=90
✓ Updating traffic... Done.
✓ Routing traffic...
Done.
URL: https://web-482915730261.asia-south1.run.app
Traffic:
  90% web-00001-k7q
  10% web-00002-p2m
$ gcloud run services update-traffic web --region asia-south1 --to-latest # promote
$ gcloud run services update-traffic web --region asia-south1 --to-revisions=web-00001-k7q=100 # roll back
```

Tags: --tag canary gives a revision its own URL to test before any traffic reaches it.

THE RULE

Ship the new revision at 0–10 %, watch, then move traffic — rollback is a routing change, not a rebuild.

Event-driven compute: a function is a Cloud Run service

WORKED EXAMPLE — AN HTTP FUNCTION

```
# main.py
import functions_framework

@functions_framework.http
def hello(request):
    name = request.args.get("name", "world")
    return {"message": f"hello, {name}"}

# requirements.txt
functions-framework==3.10.2
```

Event-driven = the platform calls you; you never poll. The function IS a Cloud Run service: same revisions, same IAM, same logs.

DEPLOY + CALL

```
$ gcloud run deploy hello --source . --function hello \
  --base-image python313 --region asia-south1 --allow-unauthenticated
...Service URL: https://hello-482915730261.asia-south1.run.app
$ curl -s 'https://hello-482915730261.asia-south1.run.app/?name=cloud'
{"message": "hello, cloud"}
```

Trigger	Fires on	Handler type
HTTP	a request to the URL	@functions_framework.http
Cloud Storage (Eventarc)	object finalized / deleted	@functions_framework.cloud_event
Pub/Sub (Eventarc)	a message on a topic	@functions_framework.cloud_event

THE RULE

Write the handler, name the trigger, pick a base image — everything between the event and your code is the platform's.

PREDICT THE OUTPUT

Two deploys, one traffic split — what does the loop print?

A service has revisions r1 (APP_VERSION=v1) and r2 (v2). You run: `update-traffic --to-revisions=r2=50,r1=50`, then curl the URL six times.

A) v1 v1 v1 v2 v2 v2

Traffic switches after three requests.

B) roughly three of each, in any order

Each request is routed independently by weight.

C) v2 six times

Latest always wins once it is healthy.

D) v1 six times

The split only applies to new clients.

Ship a container without a Dockerfile — then ship a function

HANDS-ON · LAB 2

1

Unzip the kit; the API and the function are complete — nothing to code

cd lab2/bookshelf-api · optional: python app.py and curl localhost:8080/books

2

Blank 2.1 — deploy from source, public

gcloud run deploy bookshelf-api --source _____ · then curl the URL and /books/2

3

Blank 2.2 — a second revision with APP_VERSION=v2

gcloud run deploy ... --set-env-vars _____

4

Blank 2.3 — split traffic 50/50, then curl six times

gcloud run services update-traffic ... --to-revisions=_____ (console: Revisions → Manage traffic)

5

Blank 2.4 — deploy the HTTP function

gcloud run deploy hello-function --source . --function _____ --base-image _____ ...

6

bash teardown.sh

Deletes both services

Console path

- Cloud Run → Deploy container → Service: the same choices as the flags (region, authentication, env vars)
- Service → Revisions → Manage traffic: the 50/50 split by slider
- Cloud Run → Write a function: the console form for step 4

Answers on the next slide.

THE RULE

Source in, URL out — and every deploy is a revision you can split traffic to or roll back from.

What you should have seen — and the four answers

DEBRIEF · LAB 2

ANSWERS 2.1 – 2.4

```
2.1 --source . --allow-unauthenticated
2.2 --set-env-vars APP_VERSION=v2
2.3 --to-revisions=$LATEST=50,$PREV=50
2.4 --function hello --base-image python313
```

COMMON STUMBLES

- "could not detect language" → you ran deploy outside bookshelf-api/
- revision names differ per deploy — the script reads them into \$LATEST / \$PREV; do not type them

EXPECTED — STEP 3 AND STEP 4

```
$ for i in 1 2 3 4 5 6; do curl -s $API_URL/; echo; done
{"books":4,"service":"bookshelf-api","version":"v1"}
{"books":4,"service":"bookshelf-api","version":"v2"}
{"books":4,"service":"bookshelf-api","version":"v2"}
{"books":4,"service":"bookshelf-api","version":"v1"}
{"books":4,"service":"bookshelf-api","version":"v2"}
{"books":4,"service":"bookshelf-api","version":"v1"}
$ curl -s "$FN_URL/?name=bookshelf"
{"from":"cloud-run-function","message":"hello, bookshelf"}
```

TAKE WITH YOU

- One command from source to URL; two revisions; one routing change to canary or roll back

THE RULE

The container you never wrote is still a container — pinned base, the supplied code on top, one URL in front.

05

Choosing, and the cost of hybrid

The criteria that decide between them in practice, and the bill hybrid and multi-cloud quietly hand you.

- 01 Landing in the sandbox
- 02 The compute spectrum
- 03 Compute Engine and instance groups
- 04 Containers, Kubernetes and serverless
- 05 Choosing, and the cost of hybrid**
- 06 Storage and databases
- 07 Networking: reaching and protecting the service
- 08 Artifacts and the Claude-based service

CHOOSING BETWEEN THEM

Six criteria that actually decide it — not the feature list

Criterion	Points toward VM (Compute Engine)	Points toward GKE	Points toward Cloud Run / functions
1 • State	State on local disk, long-lived sockets, licensed daemons	Stateful sets with volumes, many cooperating services	Stateless request handlers; state in a DB or bucket
2 • Traffic shape	Flat, predictable 24x7 (reserve and save)	Mixed fleet sharing capacity	Bursty, idle at night, unknown
3 • Latency budget	Always warm by definition	Always warm; you tune	Fine with min-instances; cold starts otherwise
4 • Control needed	Kernel, drivers, custom OS agents	Sidecars, custom networking, GPUs	None beyond env, CPU, memory, concurrency
5 • Team	Sysadmin skills, runbooks exist	Kubernetes fluency, platform team	Developers only; no ops team
6 • Portability	Image formats differ per cloud	Manifests port between clusters	Container ports anywhere; triggers do not

THE RULE

Answer the six honestly for the workload in front of you — the column with the most ticks is the decision.

CHOOSING BETWEEN THEM

Two anti-patterns you will be tempted by — and the plain version

ANTI-PATTERN 1

- A GKE cluster for one stateless API — "we might need Kubernetes later"
- Cost: upgrades, RBAC, networking, YAML — a platform team for one service

ANTI-PATTERN 2

- A VM for a webhook that runs 50 ms once an hour — "I know how to do it on a VM"
- Cost: 24×7 bill, patching, a process to supervise, no scale-to-zero

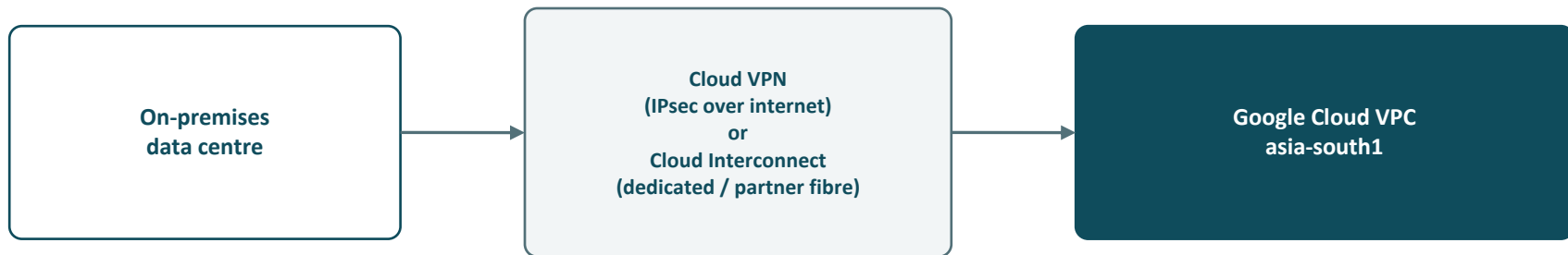
THE PLAIN VERSION (GENERIC WEB APP)

- Front end and API → Cloud Run services · nightly batch → Cloud Run job · on-upload resize → Cloud Run function
- Licensed reporting engine that needs a kernel module → the one Compute Engine VM, in a MIG of 1 for autohealing
- A cluster only when the count of services and the team's fluency both say so

THE RULE

Familiarity and ambition are not criteria — if the six say Cloud Run, the cluster can wait.

What they are — and the toolkit that connects them



GKE Enterprise: one control plane for clusters on-prem, on GCP, on other clouds (fleet, config, policy)

Legitimate reasons: data residency / regulation · an existing data centre with life left · vendor-risk policy · an acquisition on another cloud. "Avoid lock-in" alone is rarely one — see the bill on the next slide.

THE RULE

Hybrid is a network and an operating model, not a feature — budget for both before the first packet.

What they cost you — the five bills nobody puts on a slide

Bill	What it looks like	How to keep it small
Egress	Per-GB charge every time data leaves a cloud or region; chatty cross-cloud calls add latency too	Keep services that talk to each other on one side; move data in bulk, rarely
Lowest common denominator	Avoiding Spanner, BigQuery, Cloud Run to stay "portable" — running a cloud as a VM host	Portability at the container/data boundary, not the service boundary
Duplicated tooling	Two IAM models, two monitoring stacks, two CI systems, two security reviews	One control plane where possible (GKE Enterprise); shared observability
Skills and on-call	Every engineer must know both platforms; incidents span two consoles	One primary cloud; a small named team owns the exception
Consistency	No cross-cloud transactions; identity federation and DNS stitched by hand	Draw the trust and data boundaries before the first deployment

THE RULE

Every cloud you add is a second set of everything — pay it knowingly, for a reason you can write down.

ACTIVITY A

The decision matrix — four workloads, six criteria

ACTIVITY A

1 · Order-export job

Runs nightly at 02:00 for ~15 min; reads the orders DB; writes a CSV to a bucket; nothing else runs then.

2 · Licensed reporting engine

Vendor binary, needs a kernel module and a static IP tied to its licence; used by 20 analysts 9–6.

3 · Public catalogue API

Bursty: 0 req/s at night, 400 req/s at 9 am; must answer in < 300 ms; stateless; developers only, no ops team.

4 · Bookshelf: web + api

Two small stateless Python services; web is public, api must be reachable by web only; team is this room.

Criterion	Card 1	Card 2	Card 3	Card 4
1 State	stateless — CR job ✓			
2 Traffic shape				
3 Latency budget				
4 Control				
5 Team				
6 Portability				

THE RULE

Copy the grid, tick a column per criterion, and let the ticks — not the loudest opinion — make the call.

Model answers — and the one criterion that decided each

ACTIVITY A · REVEAL

Card	Decision	The deciding criterion	Runner-up and why not
1 · Order-export job	Cloud Run job on Cloud Scheduler	Traffic shape: 15 min a day → pay for 15 min	VM: bills 24 h for 15 min of work
2 · Licensed reporting engine	Compute Engine VM (MIG of 1 for autohealing)	Control: kernel module + static licence IP	GKE: privileged pods possible but a cluster for one binary
3 · Public catalogue API	Cloud Run service, min-instances 1–2	Latency: no cold starts; Team: no ops team	GKE: fluency missing; VM: bursty traffic wastes capacity
4 · Bookshelf web + api	Two Cloud Run services; api internal + IAM-only	State: stateless; Team: this room	GKE: two services do not justify a cluster (yet)

Card 4 is the app you deploy in Lab 5 — chapter 8 shows how the api stays reachable by web only.

THE RULE

One criterion usually dominates — name it, and the other five become the checklist you run before you ship.

06

Storage and databases

Buckets, classes, lifecycle rules, disks, file shares and the four databases — matched to workloads.

- 01 Landing in the sandbox
- 02 The compute spectrum
- 03 Compute Engine and instance groups
- 04 Containers, Kubernetes and serverless
- 05 Choosing, and the cost of hybrid
- 06 Storage and databases**
- 07 Networking: reaching and protecting the service
- 08 Artifacts and the Claude-based service

Buckets and objects: a flat, globally named, 11-nines object store

bucket: docs-sandbox-rohan-07 (region asia-south1)

exports/catalogue-1.txt

exports/catalogue-2.txt

images/cover-42.jpg

backups/2026-09-01.sql.gz

objects: key → bytes + metadata (content-type, storage class, generation, custom keys). "exports/" is part of the key, not a folder.

Location type	Where	Use
Region	one region (asia-south1)	app data next to compute; cheapest
Dual-region	two named regions	active-active, DR
Multi-region	a continent (asia)	serving to a wide audience

Access

- uniform bucket-level access: IAM only — always on
- signed URL: time-limited link to one object, no account needed
- public buckets: a deliberate, rare, audited choice

THE RULE

A bucket is a key-value store with a globally unique name — plan the name, the location and IAM before the first upload.

CLOUD STORAGE

Storage classes: the same bucket, priced by how often you read

Class	Minimum storage	Retrieval fee	Designed for	Availability SLA (region)
Standard	none	none	hot data, serving, short-lived	99.9 %
Nearline	30 days	yes	read about once a month: backups, recent exports	99.0 %
Coldline	90 days	yes	read about once a quarter	99.0 %
Archive	365 days	yes	read less than once a year: DR, compliance	99.0 %
Rapid	none	none	zonal, I/O-intensive AI/ML and analytics scratch	99.9 % (zone)

WORKED EXAMPLE — CREATE, THEN HAND CLASS DECISIONS TO AUTOCLASS

```
$ gcloud storage buckets create gs://docs-sandbox-rohan-07 --location=asia-south1 \  
  --default-storage-class=STANDARD --uniform-bucket-level-access \  
Creating gs://docs-sandbox-rohan-07/...  
$ gcloud storage buckets update gs://docs-sandbox-rohan-07 --enable-autoclass # let the bucket decide
```

THE RULE

Colder classes are cheaper to keep and dearer to touch — pick by read frequency, or turn on Autoclass and let the data decide.

Lifecycle rules: the bucket cleans up after itself

WORKED EXAMPLE — LIFECYCLE.JSON AND HOW IT IS APPLIED

```
{ "rule": [
  { "action": { "type": "SetStorageClass", "storageClass": "NEARLINE" },
    "condition": { "age": 30, "matchesStorageClass": ["STANDARD"] } },
  { "action": { "type": "Delete" },
    "condition": { "numNewerVersions": 3, "isLive": false } },
  { "action": { "type": "Delete" },
    "condition": { "age": 365 } }
] }
```

```
$ gcloud storage buckets update gs://docs-sandbox-rohan-07 --lifecycle-
file=lifecycle.json
Updating gs://docs-sandbox-rohan-07/...
```

Condition	Means
age	days since the object was created
matchesStorageClass	only objects currently in these classes
numNewerVersions	at least N newer versions exist
isLive	true = current version; false = old versions
matchesPrefix	key starts with... (e.g. exports/)

Rules evaluate about once a day; a transition is asynchronous. Actions cannot be undone — test on a scratch bucket first.

THE RULE

Write the retention policy of your data as rules on the bucket — not as a cron job someone forgets.

VERSIONING AND RETENTION

Versioning protects you from yourself; retention protects data from everyone

	Object versioning	Retention policy	Object hold
Protects against	accidental overwrite / delete	any delete or overwrite before the object is N old	delete of one object while the hold is set
Scope	bucket (all objects)	bucket (all objects)	one object
Reversible?	yes — turn off, delete old versions	yes until LOCKED; a locked policy is permanent	yes — release the hold
Cost	storage per kept version	none, but nothing can be deleted early	none
Command	buckets update --versioning	buckets update --retention-period=P1D	objects update --temporary-hold

DO NOT

- Lock a retention policy on anything but a compliance bucket — a locked policy cannot be shortened, and the bucket cannot be deleted until every object

THE RULE

Versioning is undo; retention is a promise — make the promise only where a regulator would ask for it.

Block, file, object: three shapes of storage for three kinds of access

Block

Persistent Disk · Hyperdisk

attached to one VM; OS formats it

boot disks, databases, anything needing a filesystem on one machine

```
gcloud compute disks create
```

File

Filestore (NFS)

mounted by many VMs / pods

shared uploads, home dirs, legacy apps needing a POSIX path

```
gcloud filestore instances create
```

Object

Cloud Storage

HTTP API from anywhere; no mount

files a service reads/writes whole; backups; static assets

```
gcloud storage cp
```

Cloud Run instances have only ephemeral memory-backed disk — they mount a bucket (Cloud Storage volumes) or an NFS share when they need files.

THE RULE

If more than one instance must see the file, it is not a disk — it is a bucket, or a share if it must be a path.

Cloud SQL, Spanner, Firestore, BigQuery — four shapes, not four sizes

Cloud SQL

Relational, one region

MySQL · PostgreSQL · SQL Server;
vertical scale, read replicas; the
default OLTP choice

e.g. orders, users, inventory

Spanner

Relational, horizontal, global

SQL with strong consistency across
regions; scales by adding nodes;
higher price floor

e.g. ledgers, global inventory, > 1
DB machine

Firestore

Document, real-time

Flexible JSON-like documents, live
listeners, mobile/offline SDKs,
serverless

e.g. user profiles, sessions, app
state

BigQuery

Analytical warehouse

Columnar, SQL over petabytes, pay
per byte scanned; not for per-
request lookups

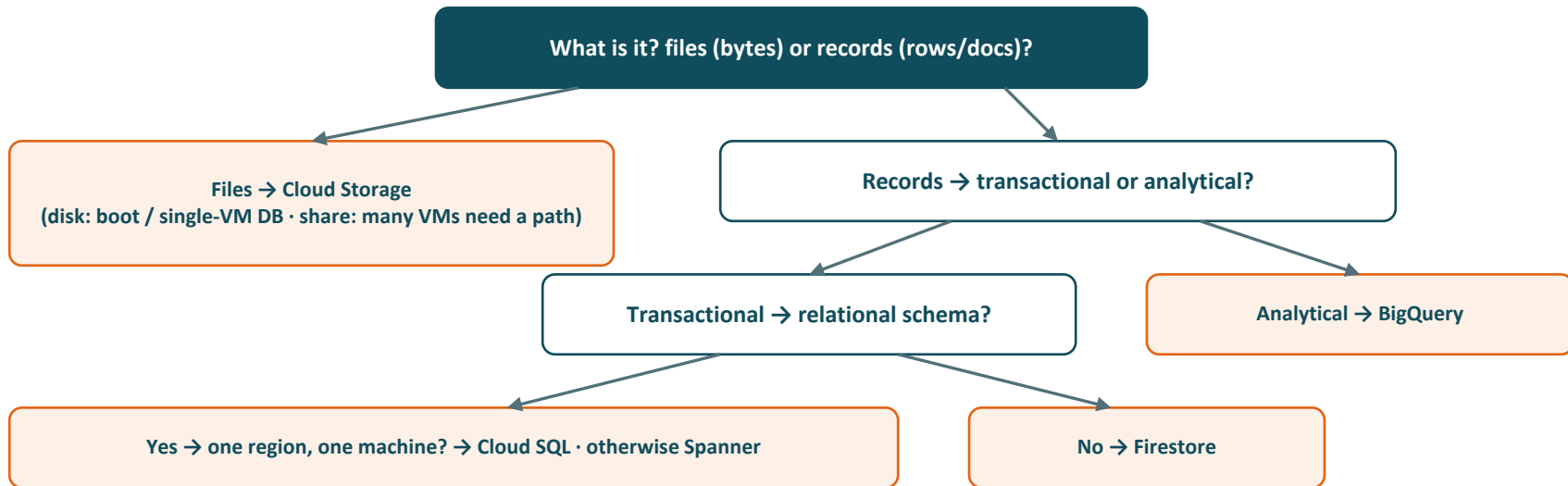
e.g. dashboards, log analysis, ML
features

Also on the shelf: AlloyDB (high-end PostgreSQL), Bigtable (wide-column, huge write throughput), Memorystore (Redis cache).

THE RULE

Transactions for one region → Cloud SQL; transactions everywhere → Spanner; documents → Firestore; questions over history → BigQuery.

One decision tree for a given workload



Overlays: Memystore (Redis) in front of hot reads · a lifecycle rule on every bucket · backups scheduled on every database.

THE RULE

Files, records, transactional, relational, one-region — five yes/no questions place any workload on the shelf.

A catalogue bucket that cleans up after itself

HANDS-ON · LAB 3

1

Step 1–2 — create the bucket, upload docs/*.txt, list them

Regional (asia-south1), STANDARD, uniform bucket-level access; name = bookshelf-docs-<your-project-id>

2

Fill the three TODOs in lifecycle.json, validate, apply (Step 3)

30 d → NEARLINE · delete non-current versions once 3 newer exist · delete everything at 365 d · python3 -m json.tool lifecycle.json

3

Step 4 — turn on versioning, overwrite catalogue-1.txt, list with -a

Two generations of the same key appear

4

Step 5 — set a 1-day retention policy, then try to delete an object

The delete fails — read the error, that is the feature

5

Step 6 — move one object to ARCHIVE by hand

What lifecycle would do for you at 365 days

6

bash teardown.sh

Clears the (unlocked) retention policy, deletes every version, deletes the bucket

THE BLANKS

```
"action": {"type": "SetStorageClass",
           "storageClass": "TODO(lab-3.3a)"},
"condition": {"age": 30, ...}
```

```
"action": {"type": "Delete"},
"condition": {"numNewerVersions":
  TODO(lab-3.3b),
              "isLive": false}
```

```
"action": {"type": "Delete"},
"condition": {"age": TODO(lab-3.3c) }
```

Answers on the next slide. Console path: Cloud Storage → Buckets → Create; bucket → Lifecycle / Protection tabs.

THE RULE

Class, lifecycle, versioning, retention — four settings, and the bucket now enforces the policy you used to write in a wiki.

What you should have seen — and the three answers

DEBRIEF · LAB 3

ANSWERS 3.3A · B · C

```
"storageClass": "NEARLINE"  
"numNewerVersions": 3  
"age": 365
```

FOR REAL DOCUMENTS

- Longer retention, locked
- Drop the 365-day delete
- Autoclass if access is unpredictable

EXPECTED — STEPS 4, 5, 6

```
$ gcloud storage ls -a $BUCKET/exports/catalogue-1.txt  
gs://bookshelf-docs-sandbox-rohan-07/exports/catalogue-1.txt#1757145601234567  
gs://bookshelf-docs-sandbox-rohan-07/exports/catalogue-1.txt#1757145688765432  
$ gcloud storage rm $BUCKET/exports/catalogue-2.txt  
ERROR: (gcloud.storage.rm) HTTPError 403: Object 'bookshelf-docs-sandbox-rohan-07/exports/catalogue-2.txt'  
is subject to bucket's retention policy and cannot be deleted, overwritten or  
archived until 2026-09-07T...  
--> blocked by retention, as expected  
$ gcloud storage ls -L $BUCKET/exports/catalogue-3.txt | grep -i 'storage class'  
Storage Class: ARCHIVE
```

COMMON STUMBLER

- 409 on create: bucket names are global — the script's project-id suffix must be intact

THE RULE

The bucket said no to its Owner — that is the whole point of a retention policy.

ACTIVITY B

Pick the store — five workloads, one service each

ACTIVITY B

#	Workload	Your pick	Why (one line)
1	Product images uploaded by sellers, served on every page view, kept forever		
2	Orders and payments for one country; strict consistency; 2 000 writes/s at peak		
3	"Revenue by category by hour, last 3 years" for the finance dashboard		
4	Per-user shopping carts with live updates on web and mobile		
5	Bookshelf's catalogue — 4 books today, maybe 4 000 next year; read by the api on every request		

Copy the table; fill the last two columns. Answers on the next slide.

THE RULE

Run the five questions, write one line of reasoning — the line is what survives the code review.

Model answers

ACTIVITY B · REVEAL

#	Workload	Pick	Why
1	Seller product images, served every page view	Cloud Storage, Standard (multi-region if global) + Cloud CDN	Files; hot; served over HTTP — never on a disk
2	Orders and payments, one country, strict consistency	Cloud SQL (PostgreSQL)	Relational, transactional, one region; Spanner only for multi-region writes
3	Revenue by category by hour, 3 years	BigQuery	Analytical scan over history; not per-request
4	Per-user carts, live on web and mobile	Firestore	Document per user; real-time listeners; offline SDKs
5	Bookshelf catalogue read by the api	Cloud SQL (+ Memorystore later)	Relational and tiny; api stays stateless; today's in-memory list is the placeholder

THE RULE

If two people picked differently and both can say why, the workload description was missing a constraint — write it down.

07

Networking: reaching and protecting the service

VPCs, subnets, firewall rules, routes, load balancers, private access and DNS: who can reach what, and how.

- 01 Landing in the sandbox
- 02 The compute spectrum
- 03 Compute Engine and instance groups
- 04 Containers, Kubernetes and serverless
- 05 Choosing, and the cost of hybrid
- 06 Storage and databases
- 07 Networking: reaching and protecting the service**
- 08 Artifacts and the Claude-based service

VPC NETWORKS

A VPC is global; its subnets are regional; its IPs are yours to plan

VPC: prod-vpc (global) — routes · firewall rules · peering

subnet asia-south1
10.10.0.0/24

subnet europe-west1
10.20.0.0/24

subnet us-central1
10.30.0.0/24

vm 10.10.0.2

vm 10.10.0.3

vm 10.20.0.2

vm 10.30.0.2

VMs in different regions talk over internal IPs — one network, no VPN, no peering.

Auto mode

a subnet in every region from 10.128.0.0/9; the 'default' network

Custom mode

no subnets until you create them; you choose every range — production

THE RULE

One network per environment, one subnet per region you actually use, and ranges you chose on purpose.

IP ADDRESS PLANNING

CIDR, reserved addresses, and ranges that will not collide later

WORKED EXAMPLE — HOW MANY VMS FIT IN A /24

```
10.10.0.0/24 → 256 addresses
- 10.10.0.0    network address
- 10.10.0.1    default gateway
- 10.10.0.254  reserved (second-to-last)
- 10.10.0.255  broadcast
= 252 usable VM addresses

/22 = 1 024 (1 020 usable)   /20 = 4 096 (4 092 usable)
```

External IPs are a separate pool: ephemeral by default; reserve a static one only when DNS or a licence needs it (Lab 4 uses none).

Plan	Range	Purpose
prod	10.10.0.0/16	environment
prod / asia-south1 / apps	10.10.0.0/22	1 020 VMs
prod / asia-south1 / data	10.10.4.0/24	databases
gke pods (secondary)	10.11.0.0/17	32 k pods
staging	10.20.0.0/16	never overlaps prod

THE MISTAKE THAT CANNOT BE FIXED

- Two networks with overlapping ranges can never be peered or VPN-connected — renumbering means rebuilding

THE RULE

Write the address plan before the first subnet — ranges are the one thing in cloud you cannot rename later.

FIREWALL RULES

Stateful, network-scoped, matched by priority — and targeted by tag or identity

WORKED EXAMPLE — SSH ONLY FROM GOOGLE'S IAP RANGE

```
$ gcloud compute firewall-rules create allow-iap-ssh \
  --network=prod-vpc --direction=INGRESS --action=ALLOW \
  --rules=tcp:22 --source-ranges=35.235.240.0/20 --target-tags=private-vm
Creating firewall...done.
NAME          NETWORK  DIRECTION  PRIORITY  ALLOW  DENY  DISABLED
allow-iap-ssh  prod-vpc  INGRESS    1000      tcp:22  False
```

Implied rule (every VPC)	Priority	Effect
deny all ingress	65535	nothing reaches a VM unless a rule allows it
allow all egress	65535	VMs may reach anything (routes permitting)

Field	Options	Lab 4 value
direction	INGRESS · EGRESS	INGRESS
action / priority	ALLOW · DENY / 0–65535 (lower wins)	ALLOW / 1000
source	CIDRs · source tags · source service accounts	35.235.240.0/20 (IAP)
target	all · --target-tags · --target-service-accounts	tag private-vm
rules	protocol:ports	tcp:22

THE RULE

Default-deny in, default-allow out — every open port is a rule you wrote, so make its source as narrow as the job allows.

ROUTES

Routes decide where a packet goes next — the default one points at the internet

WORKED EXAMPLE — WHAT A FRESH CUSTOM VPC WITH ONE SUBNET HAS

```
$ gcloud compute routes list --filter="network:prod-vpc"
```

NAME	NETWORK	DEST_RANGE	NEXT_HOP	PRIORITY
default-route-3f1a9c0b2e7d4a61	prod-vpc	10.10.0.0/24	prod-vpc	0
default-route-9b8e2d4c6a1f0e37	prod-vpc	0.0.0.0/0	default-internet-gateway	1000



Need	Fix
Internet from private VMs (apt, pip, webhooks)	Cloud NAT on the subnet — managed, no VM to run
Google APIs from private VMs (storage, run, ...)	Private Google Access on the subnet — next slides

THE RULE

No external IP means no internet by default — add Cloud NAT for the world, Private Google Access for Google, and nothing else.

LOAD BALANCING

The load-balancer family: Application vs Network, external vs internal, global vs regional

	External (internet-facing)	Internal (from your VPC)
Application LB (L7 · HTTP/S)	Global (anycast IP, multi-region backends, Cloud CDN, Cloud Armor) · Regional · Classic	Regional · Cross-region
Proxy Network LB (L4 · TCP/SSL proxy)	Global · Regional · Classic	Regional · Cross-region
Passthrough Network LB (L4 · TCP/UDP, source IP kept)	Regional (Global in preview)	Regional

CHOOSE THE APPLICATION LB WHEN

- HTTP/S, URL- or host-based routing, TLS at the edge, CDN, WAF (Cloud Armor), Cloud Run or MIG backends

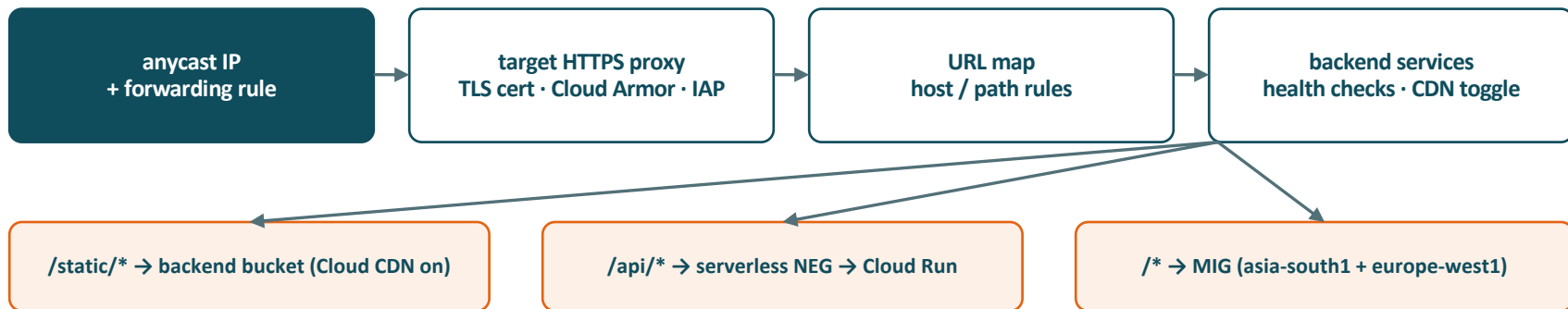
CHOOSE A NETWORK LB WHEN

- Non-HTTP protocols, UDP, TLS passthrough to the backend, the client's source IP must be preserved

THE RULE

For an HTTP service the answer is almost always the global external Application Load Balancer — everything else is an exception you can name.

Anatomy of a global external Application Load Balancer, with Cloud CDN

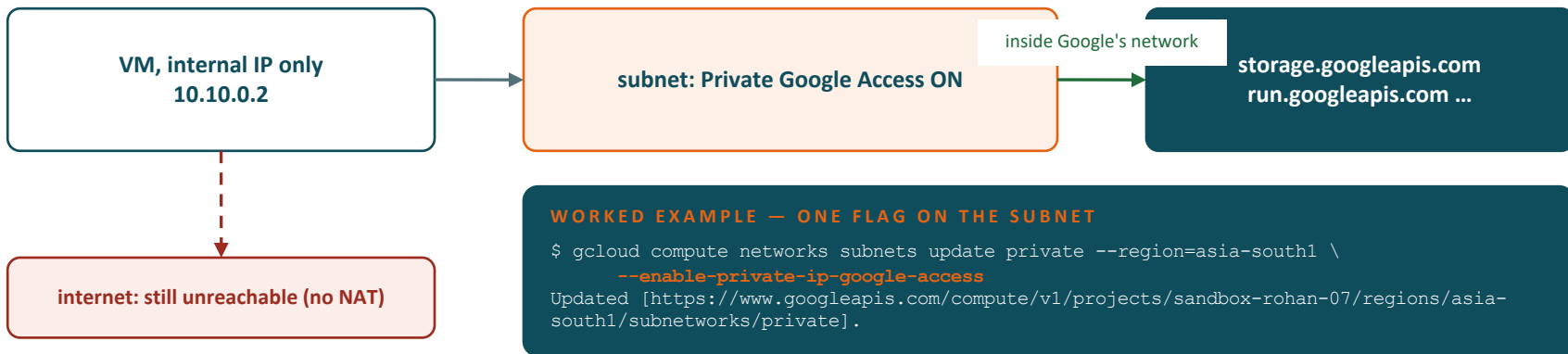


Add-on	Where	Gives you
Cloud CDN	per backend	edge caching of cacheable responses; cache keys, TTL overrides, invalidation
Cloud Armor	at the proxy	WAF rules (OWASP), rate limiting, IP/geo allow/deny
Identity-Aware Proxy	at the proxy	Google-identity sign-in in front of internal tools — no VPN

THE RULE

One IP, one certificate, one URL map — and the cache, the WAF and the sign-in page are checkboxes on it, not servers you run.

Private Google Access: reach Google APIs with no external IP

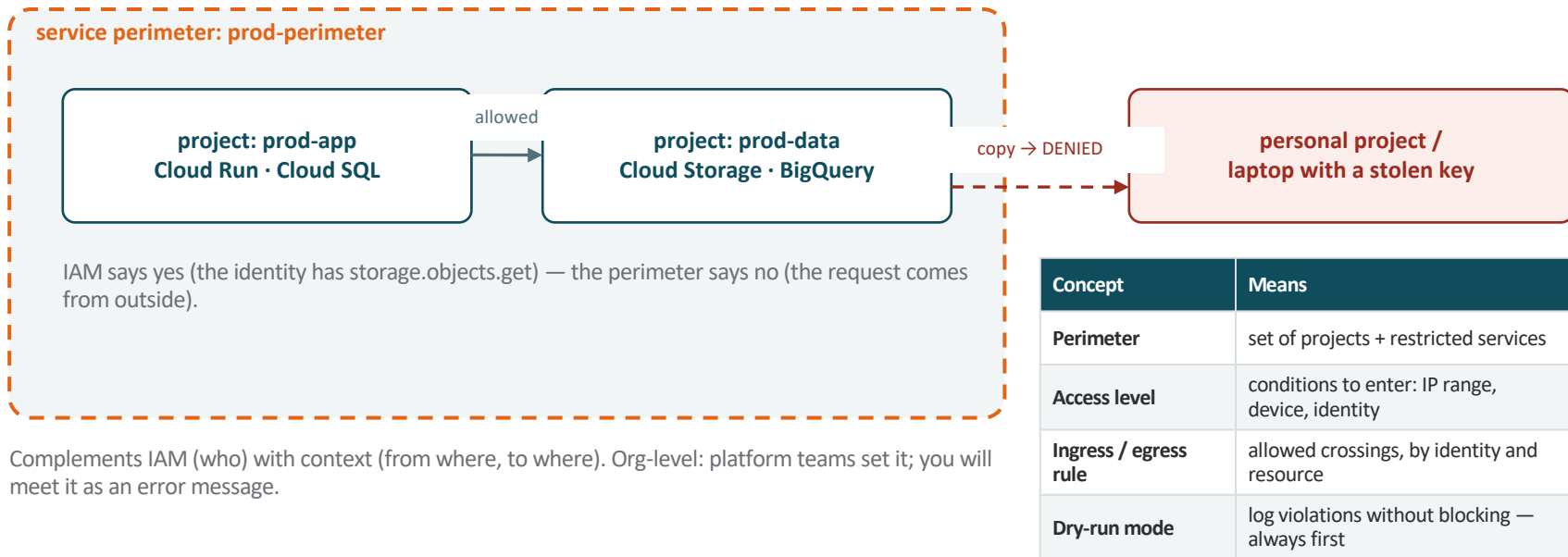


Option	What it is	When
Private Google Access	subnet flag; default route; public API IPs delivered privately	the default choice — Lab 4
private/restricted.googleapis.com	fixed VIPs + private DNS zone; restricted = VPC-SC-supported APIs only	no default route, or perimeter enforcement
Private Service Connect	an endpoint IP in your subnet for Google APIs or a producer's service	hybrid, strict egress, partner services

THE RULE

Turn Private Google Access on for every subnet with internal-only VMs — it is free, and it is the difference between "unreachable" and "works".

VPC Service Controls: a perimeter around the data, on top of IAM



Complements IAM (who) with context (from where, to where). Org-level: platform teams set it; you will meet it as an error message.

THE RULE

IAM decides who; a VPC Service Controls perimeter decides from where and to where — you need both to stop a stolen key.

Cloud DNS zones, and the four ways a service is allowed to be reached

WORKED EXAMPLE — A PUBLIC ZONE, AN A RECORD, A PRIVATE ZONE

```
$ gcloud dns managed-zones create example-public --dns-name=example.com. --description="public zone"
Created [https://dns.googleapis.com/dns/v1/projects/sandbox-rohan-07/managedZones/example-public].
$ gcloud dns record-sets create api.example.com. --zone=example-public --type=A --ttl=300 --rrdatas=34.120.45.67
NAME          TYPE  TTL  DATA
api.example.com. A      300  34.120.45.67
$ gcloud dns managed-zones create corp-internal --dns-name=internal.example. --visibility=private --networks=prod-vpc
```

Exposure	How	Use for
Public, anonymous	Cloud Run --allow-unauthenticated / LB with no auth	marketing site, public API — deliberately
Public, protected	global external ALB + Cloud Armor (WAF, rate limits) + TLS	customer-facing apps
Identity at the edge	Identity-Aware Proxy on the LB or Cloud Run	internal tools: no VPN, sign in with Google identity
Internal only	Cloud Run --ingress=internal · internal LB · no external IP	backends called only by other services — the Bookshelf api

THE RULE

Give every service the narrowest rung of the ladder that still lets its real callers in — and write which rung it is on.

YOUR CALL

A VM with no external IP runs `pip install` and `gcloud storage ls` — what happens?

Subnet: custom VPC, Private Google Access OFF, no Cloud NAT. Both commands time out. You turn Private Google Access ON. Now?

A) Both work

PGA gives the VM a way out.

B) Neither works

Only an external IP or NAT gives a way out.

C) gcloud storage works,

pip does not

PGA covers Google APIs only; PyPI is the internet.

D) pip works, gcloud

storage does not

PGA is for package mirrors.

A private subnet that can still reach Google

HANDS-ON · LAB 4

1

Step 1 — create a custom-mode VPC bookshelf-vpc

No subnets until you say so

2

TODO 4.2 — one subnet in asia-south1 with a /24 inside 10.0.0.0/8

How many usable addresses is that?

3

TODO 4.3 — firewall: SSH (tcp:22) only from 35.235.240.0/20 (IAP) to tag private-vm

Nothing else is open on this network

4

Step 4 — list the routes the VPC got for free

One local route per subnet, one default route to the internet gateway

5

TODO 4.5 — a VM with NO external IP; Step 6 — SSH in through IAP

gcloud compute ssh --tunnel-through-iap runs two curls: internet and the Storage API both fail

6

TODO 4.7 — turn on Private Google Access; Step 8 — rerun the curls; teardown

APIs now answer, the internet still does not

THE BLANKS 4.2 · 4.3 · 4.5 · 4.7

```
--range=_____  
  
--rules=tcp:_____ \  
--source-ranges=_____  
  
--subnet=bookshelf-private _____  
  
subnets update ... _____
```

Answers on the next slide. Console path: VPC network → Create VPC (custom); Firewall → Create rule; VM → Create (Networking: no external IP); subnet → Edit → Private Google Access On.

THE RULE

Custom VPC, narrow firewall, no external IP, Private Google Access — that is the shape of every production subnet.

What you should have seen — and the four answers

DEBRIEF · LAB 4

ANSWERS 4.2 · 4.3 · 4.5 · 4.7

```
--range=10.10.0.0/24          # 252 usable  
--rules=tcp:22 --source-ranges=35.235.240.0/20  
--no-address  
--enable-private-ip-google-access
```

TAKE WITH YOU

- No external IP + PGA + Cloud NAT (if needed) = the production subnet shape
- IAP TCP forwarding replaces bastion hosts

EXPECTED — STEPS 6 AND 8

```
# step 6 (before PGA)  
internet: unreachable (expected)  
storage api: unreachable (expected)  
# step 8 (after PGA)  
internet: unreachable (expected)  
storage api: HTTP 400  
gs://bookshelf-docs-sandbox-rohan-07/
```

COMMON STUMBLER

- SSH hangs: the IAP firewall rule is missing or has the wrong range
- Subnet delete fails: the VM is still there — teardown order

THE RULE

HTTP 400 from a Google API was the best news of the lab — the private path works.

08

Artifacts and the Claude-based service

Where build outputs live, how the two services find and trust each other, and where a Claude-based service sits.

- 01 Landing in the sandbox
- 02 The compute spectrum
- 03 Compute Engine and instance groups
- 04 Containers, Kubernetes and serverless
- 05 Choosing, and the cost of hybrid
- 06 Storage and databases
- 07 Networking: reaching and protecting the service
- 08 Artifacts and the Claude-based service**

ARTIFACT REGISTRY

Where build outputs live: one registry for images, packages and more

THE IMAGE URL YOU WILL TYPE IN LAB 5

`asia-south1-docker.pkg.dev / sandbox-rohan-07 / bookshelf / bookshelf-api : 1.0`
└─ region + format host └─ project └─ repo └─ image └─ tag (or @sha256:...)

Mode	What it does	Use
Standard	you push artifacts; you own them	your images and packages — Lab 5
Remote	caches an upstream (Docker Hub, PyPI, npm, Maven...) on first pull	private VMs and builds with no internet; pin what you depend on
Virtual	one endpoint that searches several repos in order	one URL for developers: private first, then the cache

Also in the box

- Formats: Docker/OCI, Helm, Python, npm, Maven, Go, apt/yum, generic
- Cleanup policies: delete untagged after N days, keep the last 10 versions
- Artifact Analysis: vulnerability scanning of pushed images
- IAM per repository: reader vs writer

Container Registry (gcr.io) shut down for writes on 18 Mar 2025 — every tutorial that says gcr.io is out of date.

THE RULE

Every deployable is an artifact with a region, a repo, a name and an immutable digest — build once, store once, deploy by reference.

Worked example: Cloud Build builds it, the registry keeps it, Cloud Run pulls it

WORKED EXAMPLE — BUILD AND PUSH WITH CLOUD BUILD

```
$ gcloud artifacts repositories create bookshelf --repository-format=docker --location=asia-south1
Created repository [bookshelf].
```

```
$ gcloud builds submit api --tag asia-south1-docker.pkg.dev/sandbox-rohan-07/bookshelf/bookshelf-api:1.0
```

```
Creating temporary archive of 4 file(s) totalling 2.1 KiB before compression.
```

```
Uploading tarball of [api] to [gs://sandbox-rohan-07_cloudbuild/source/1757150312.44-alb2c3.tgz]
```

```
Created [https://cloudbuild.googleapis.com/v1/projects/sandbox-rohan-07/locations/global/builds/7f3e...].
```

```
Step #0: Pulling image: gcr.io/cloud-builders/docker
```

```
Step #0: Step 1/9 : FROM python:3.13-slim
```

```
Step #0: ... Successfully built 9c2a4f1d7e8b
```

```
PUSH Pushing asia-south1-docker.pkg.dev/sandbox-rohan-07/bookshelf/bookshelf-api:1.0
```

```
1.0: digest: sha256:4b7c3e2f9a1d8e6c5b4a3f2e1d0c9b8a7f6e5d4c3b2a1f0e9d8c7b6a5f4e3d2c size: 1786
```

ID	CREATE_TIME	DURATION	SOURCE	IMAGES	STATUS
7f3e...	2026-09-06T10:25:12+00:00	1m4s	gs://.../1757150312.tgz	asia-south1-docker.pkg.dev/.../bookshelf-api:1.0	SUCCESS

Tag = a movable label (1.0, latest). Digest = the immutable identity. Deploy by tag in labs; pin the digest in production.

THE RULE

No Docker on the laptop, no image on the laptop — the build runs in the cloud and the artifact never leaves the region.

SERVICE TO SERVICE

Two services, one private: identity, ingress and the VPC path between them



WORKED EXAMPLE — THE CALLER MINTS AN ID TOKEN; THE CALLEE'S IAM ALLOWS EXACTLY ONE CALLER

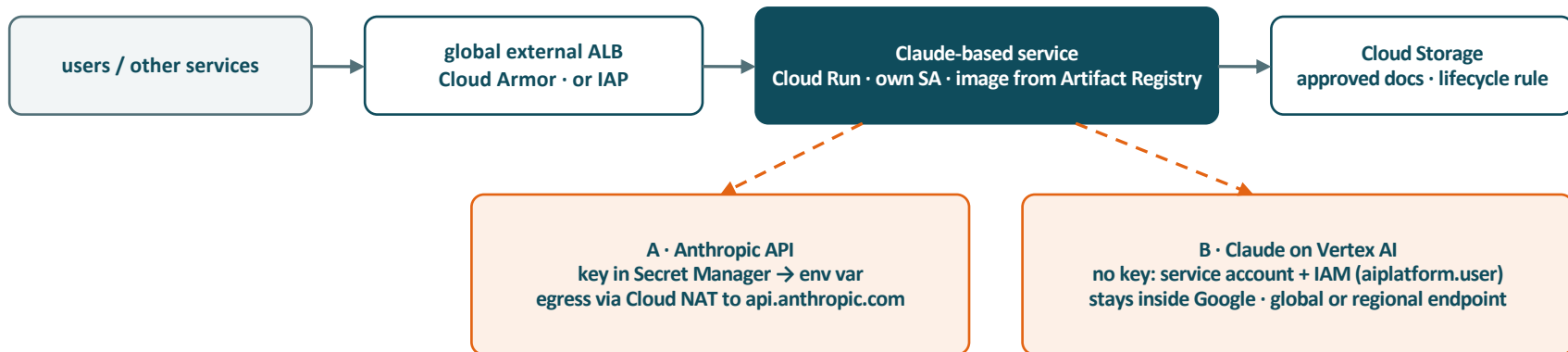
```
def api_headers():  
    auth_req = google.auth.transport.requests.Request()  
    token = google.oauth2.id_token.fetch_id_token(auth_req, API_URL)    # audience = api URL  
    return {"Authorization": f"Bearer {token}"}  
  
$ gcloud run services add-iam-policy-binding bookshelf-api \  
    --member=serviceAccount:bookshelf-web-sa@sandbox-rohan-07.iam.gserviceaccount.com --role=roles/run.invoker
```

THE RULE

Private = internal ingress + required authentication + one named caller — three settings, and no secret was created.

WHERE A CLAUDE-BASED SERVICE SITS

The reference architecture — and the two ways the model is reached



	A · Anthropic API	B · Claude on Vertex AI
Credential	API key (Secret Manager)	service account, IAM role — no key
Network path	internet egress (NAT)	Google-internal; Private Google Access works
Residency / billing	Anthropic's regions; Anthropic invoice	global (no premium) or us/eu/regional (+10 %); GCP invoice
SDK	<code>anthropic.Anthropic()</code>	<code>anthropic.AnthropicVertex(project_id, region)</code>

THE RULE

Same Cloud Run shape as Bookshelf; only the model path differs — a key in Secret Manager, or a service account and Vertex AI.

Ship the two-service app — build, store, deploy, restrict, connect

HANDS-ON · LAB 5

1

Unzip; both services are complete — nothing to code

Optional local check: run api on :8091 and web on :8090, curl localhost:8090 → 4 books

2

Blank 5.1 — create the Artifact Registry repo; Step 2 — Cloud Build both images

--repository-format=_____ · gcloud builds submit api --tag \$AR/bookshelf-api:1.0 (and web)

3

Step 3 — two service accounts; Blank 5.4 — deploy the api private

--ingress _____ · curl from Cloud Shell → 404

4

Blank 5.5 — grant ONLY web-sa the right to call the api

add-iam-policy-binding bookshelf-api --member=serviceAccount:... --role=_____

5

Blank 5.6 — PGA on the subnet; deploy web through the VPC; open the URL

--network default --subnet default --vpc-egress _____ --set-env-vars API_URL=...

6

bash teardown.sh

Also removes Lab 2's source-deploy repo

Console path

- Artifact Registry → Create repository (Docker, asia-south1)
- Cloud Run → Deploy container → Security: Require authentication · Networking: Ingress Internal
- Service → Permissions → Add principal → Cloud Run Invoker
- Deploy web → Networking → Connect to a VPC, send all traffic

Answers on the next slide.

THE RULE

Build once into the registry, run each service as its own identity, let exactly one caller in — that is a production deployment.

What you should have seen — and where each resource sits on the map

DEBRIEF · LAB 5

ANSWERS 5.1 · 5.4 · 5.5 · 5.6

```
5.1 --repository-format=docker
5.4 --ingress internal --no-allow-unauthenticated
5.5 --role=roles/run.invoker
5.6 --vpc-egress all-traffic
```

IF THE PAGE SAYS...

- 403 → the run.invoker binding (5.5) is missing or on the wrong service account
- 404 → web's egress is not going through the VPC (5.6), or PGA is off on the subnet
- connection error → API_URL env var not set on web

EXPECTED — STEPS 4 AND 6

```
$ curl -s -o /dev/null -w "from Cloud Shell: HTTP %{http_code}\n"
$API_URL/
from Cloud Shell: HTTP 404
$ gcloud run services describe bookshelf-web --
format='value(status.url)'
https://bookshelf-web-482915730261.asia-south1.run.app
$ curl -s $WEB_URL/ | grep -o "<li>.*</li>" | head -4
<li><b>The Pragmatic Programmer</b> — Hunt & Thomas (1999)</li>
<li><b>Designing Data-Intensive Applications</b> — Kleppmann
(2017)</li>
<li><b>Site Reliability Engineering</b> — Beyer et al. (2016)</li>
<li><b>Release It!</b> — Nygard (2018)</li>
```

NEVER CREATED TODAY

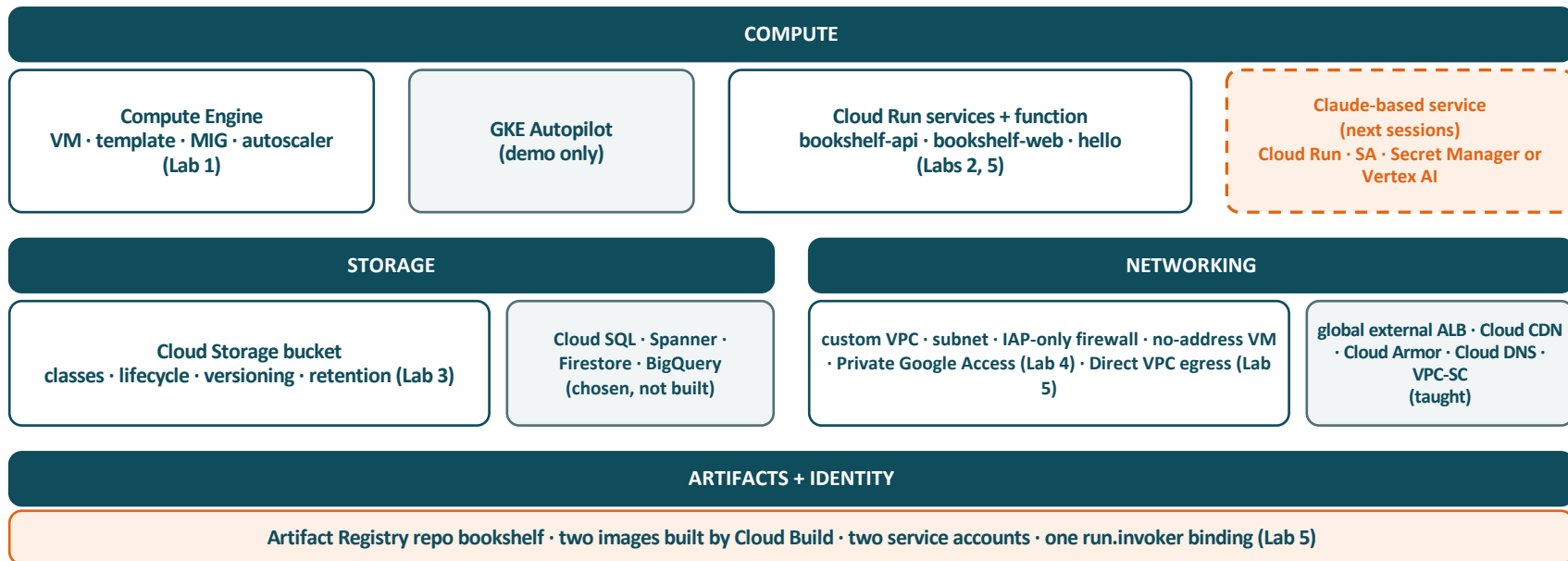
- a key · a password · a public VM IP · a cluster · a line of code

THE RULE

Same URL, two answers: the request's path through the network — not the URL — decided who got in.

THE LANDSCAPE IN ONE PICTURE

Where today's five labs landed — and the slot for the Claude-based service



BEFORE YOU LEAVE

Tear it all down — and what it would have cost to forget

- 1** **bash lab5/teardown.sh**
both Cloud Run services, both service accounts, the bookshelf repo, the cloud-run-source-deploy repo from Lab 2
- 2** **bash lab4/teardown.sh**
private-vm → firewall rule → subnet → bookshelf-vpc (order matters)
- 3** **bash lab3/teardown.sh**
clears the retention policy, deletes every object version, deletes the bucket
- 4** **bash lab2/teardown.sh · bash lab1/teardown.sh**
if you skipped them earlier
- 5** **Verify**
gcloud compute instances list · gcloud run services list · gcloud storage ls · gcloud compute networks list → empty (only 'default' remains)

Left running overnight	Roughly
3 small VMs (Lab 1)	≈ US\$1 / day
an idle GKE Autopilot cluster	≈ US\$2–3 / day
Cloud Run at zero traffic	0
a bucket with a few KB	≈ 0
a repo with a few images	cents

Prices are order-of-magnitude, on the build date, and change — the habit does not.

THE RULE

A lab is finished when the resource list is empty — not when the page loads.

WHAT COMES NEXT

You now have the map. The next sessions put the Claude-based service on it.

- Compute: it is a Cloud Run service — you deployed two today, with identity and private ingress.
- Storage: its approved documents live in a bucket with the rules you wrote in Lab 3.
- Networking: it reaches Google privately the way Lab 4's VM did; the outside reaches it through the ladder from chapter 7.
- Artifacts: it is an image in the registry, built by Cloud Build, deployed by reference — exactly like bookshelf-api.